LLM Consortium for Software Design Refinement:

A Controlled Experiment on Multi-Agent Collaboration Topologies

Nagarjuna Kanamarlapudi

Praveen Kanamarlapudi

Abstract

We present a controlled experiment evaluating 13 multi-agent LLM collaboration topologies for software architecture design. Using a 2×2×2 factorial design (Authority × Roles × Dynamics), we conducted 520 experimental runs across 8 design tasks of varying complexity, with 5 repetitions each. Designs were evaluated on a 12-dimensional rubric by three independent automated evaluators (GPT-OSS 120B, Claude Opus 4.6, Claude Sonnet 4.6). We report four core findings. ***First, cross-model review wins unanimously***—generate with one model, review with another—ranking #1 by all three evaluators (weighted ensemble: 4.740/5.0). ***Second, structural adversarial (v4b) ranks #2 (ensemble: 4.637)***, rescuing the original v4 null result through prompt engineering that demands rewrite mandates rather than patches. ***Third, evaluator diversity is itself a finding***—all three evaluators agree v2b is best and v3 is worst, but disagree sharply on v4b (Claude d=1.44 vs. GPT-OSS d=0.45), revealing how different model families weight design qualities. ***Fourth, parallel merge is fundamentally broken***—all three evaluators place merge variants in the bottom tier (3.65–3.79), due to token starvation and the Frankenstein effect. The weighted ensemble (2×Opus + 2×Sonnet + 1×GPT-OSS) provides robust rankings across 520 runs, confirmed through independent cross-validation.

Keywords: *Large Language Models, Multi-Agent Systems, Software Architecture, Collaboration Topologies, Cross-Model Review, Empirical Software Engineering*

I. INTRODUCTION

The application of Large Language Models (LLMs) to software engineering has progressed from code completion to architectural design generation. A natural question arises: can multiple LLM agents, collaborating through structured workflows, produce higher-quality software designs than a single model iterating alone?

We formalize this question through a 2×2×2 factorial design crossing three dimensions of multi-agent collaboration: Authority (centralized vs. decentralized), Roles (homogeneous vs. specialized), and Dynamics (cooperative vs. adversarial). This yields eight theoretical cells, from which we derive 13 concrete variant configurations spanning the practical design space.

Our experiment comprises 520 controlled runs (13 variants × 8 design tasks × 5 repetitions) evaluated on a 12-dimensional application design rubric by three independent evaluator models. The tasks range from simple (URL shortener) to complex (collaborative editor, SaaS billing), enabling analysis of how collaboration topology interacts with problem difficulty. Rankings are reported using a weighted ensemble (2×Opus + 2×Sonnet + 1×GPT-OSS) that prioritizes the most recent architectural understanding from Claude's model family while maintaining independent signal diversity from OpenAI.

The central finding is both simple and surprising: the most effective multi-agent workflow is also the simplest—**generate with one model, review with a different model.** This cross-model review (v2b) outperforms every sophisticated multi-agent topology tested, including specialist panels, rotating leadership, structured debate, and parallel generation with merge. The mechanism we identify—complementary blind spots arising from different training distributions—has direct practical implications for LLM-based software engineering workflows.

This paper makes six contributions: (1) a systematic empirical comparison of 13 multi-agent topologies ranked by a weighted ensemble (2×Opus + 2×Sonnet + 1×GPT-OSS), (2) identification and empirical validation of the complementary blind spots mechanism in cross-model review, which ranks #1 unanimously across all three evaluators, (3) a trace-level autopsy of why parallel merge is fundamentally broken (token starvation, Frankenstein effect) and why the original adversarial topology (v4) produced null results, (4) a redesigned adversarial variant (v4b) demonstrating that prompt engineering demanding structural rethinking rather than patching can rescue a null result to #2 ensemble ranking, (5) discovery that evaluator disagreement reveals systematic differences in how model families weight design qualities (complementary blind spots extends to evaluation), and (6) three-evaluator cross-validation confirming that v2b cross-model review and v3 merge failure are robust findings across all evaluators.

II. RELATED WORK

Multi-agent LLM systems have been explored

across several domains. ChatDev and MetaGPT demonstrated that role-playing LLM agents can collaborate on software development tasks, though without controlled comparison of topologies. AgentCoder used a generate-test-debug loop for code generation, while MapCoder explored multi-agent planning for competitive programming problems.

In the broader multi-agent literature, Du et al. showed that LLM debate can improve mathematical reasoning, and Liang et al. demonstrated that diverse agents improve creative writing quality. However, these studies typically compare a single multi-agent configuration against a single-agent baseline, rather than systematically exploring the design space of collaboration topologies.

Our work differs in three key ways. First, we use a factorial experimental design to isolate the effects of authority, roles, and dynamics independently. Second, we evaluate on a real-world software engineering task (application architecture design) rather than synthetic benchmarks. Third, we provide trace-level analysis of why specific topologies succeed or fail, moving beyond aggregate quality comparisons.

The cross-model review finding connects to ensemble learning literature, where diversity among base learners is known to improve ensemble performance. Our contribution is showing that this principle extends to LLM-based review: a different model's training distribution provides epistemic diversity that manifests as complementary blind spots in design review.

III. METHODOLOGY

A. Experimental Design

We adopt a 2×2×2 factorial framework crossing Authority (centralized vs. decentralized), Roles (homogeneous vs. specialized), and Dynamics (cooperative vs. adversarial). This theoretical framework yields eight cells, from which we derive 13 executable variant configurations, including subvariants that explore specific mechanisms within a cell (e.g., same-model vs. cross-model review within the centralized-homogeneous-cooperative cell).

B. Variant Configurations

The 13 variants tested are organized into six families. The Baseline family includes v1 (self-refine with 3-round evaluator loop) and v1a (single-shot, no refinement). The Leader+Reviewer family includes v2a (same-model review), v2b (cross-model review with openai/gpt-oss-120b-maas as reviewer), and v2c (multi-model panel with two GPT-OSS reviewers). The Parallel Merge family includes v3a (naive merge), v3b (rubric-guided merge), and v3c (dialectical merge). The Adversarial family includes v4 (binary accept/reject gate) and v4b (structural adversarial with principal-architect-level review demanding rewrite mandates, 5 rounds). The remaining variants are v5 (specialist panel with security/scalability/maintainability experts), v6 (rotating leader), and v8 (structured debate with judge).

Fig. 1 presents the workflow topology for each major variant family, showing how agents pass designs, reviews, and feedback to each other.

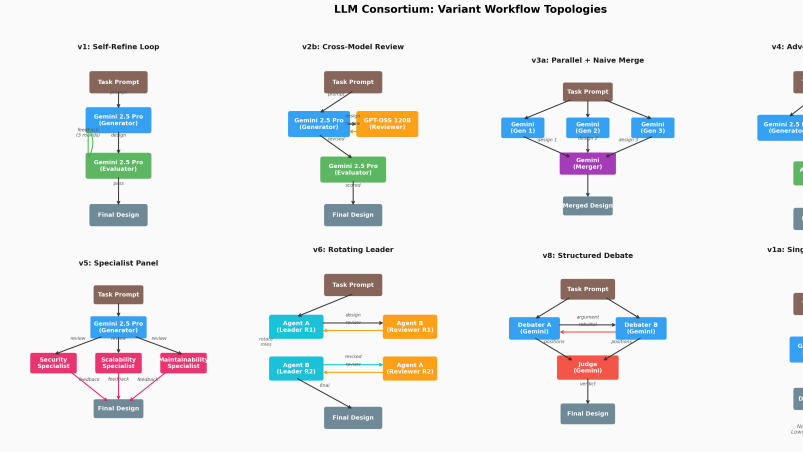

Figure 1: fig7_variant_workflows.png

Fig. 1. Workflow topologies for the eight major variant families. Arrows indicate design/review/feedback flow between agents. Colors distinguish roles: blue=generator, orange=reviewer, green=evaluator, purple=merger, red=adversarial/judge, cyan=leader, pink=specialist.

C. Design Tasks

Eight software design tasks of varying complexity were used: T1 URL Shortener [Simple], T2 Rate Limiter [Simple], T3 Notification System [Medium], T4 Task Queue [Medium], T5 SaaS Billing [Complex], T6 Collaborative Editor [Complex], T7 Access Control [Complex], and T8 Order Processing [Complex]. Each task requires a complete application architecture design including domain model, module structure, API contracts, error handling, and non-functional considerations.

D. Evaluation Framework

Each design was independently evaluated by three model families: openai/gpt-oss-120b-maas,

claude-opus-4-6, and claude-sonnet-4-6, using the same 12-dimension rubric. To produce robust quality rankings, we compute a weighted ensemble: Ensemble = (2×Opus + 2×Sonnet + 1×GPT-OSS) / 5. The weighting reflects two considerations: (1) the Claude models represent more recent architectural understanding, and (2) having two independent Claude evaluators (Opus and Sonnet) that agree strengthens confidence in their signal. All rankings and findings in this paper use the weighted ensemble unless stated otherwise.

The rubric evaluates designs on 12 dimensions: Requirements/Scope Clarity, Domain Model Quality, Module Structure & Dependency Direction, Interface & Contract Design, Data Flow Clarity, Error Handling Rigor, Testability, State Management & Lifecycle, Concurrency Safety, Cross-Cutting Concerns, Design Decision Justification, and Right-Sizing/Simplicity. Each dimension is scored 1–5 by the automated evaluators with the rubric in-context.

E. Models and Infrastructure

Design generation uses gemini-2.5-pro via Google Vertex AI. Cross-model review and primary evaluation use openai/gpt-oss-120b-maas via the Vertex OpenAI compatibility layer. All 520 runs (480 original + 40 v4b) completed successfully. Independent re-evaluations of all designs were performed by claude-opus-4-6 and claude-sonnet-4-6 to enable three-evaluator cross-validation.

IV. RESULTS

A. Weighted Ensemble Rankings

The global Friedman test confirms that variant choice has a statistically significant effect on design quality ($\chi^2 = 220.3$, $p < 10^{-40}$). Table I presents the complete variant ranking using the weighted ensemble (2×Opus + 2×Sonnet + 1×GPT-OSS) as the primary scoring system, with per-evaluator scores and cost.

TABLE I: Weighted Ensemble Rankings with Per-Evaluator Scores

#	Variant	Ensemble	GPT-OSS	Opus	Sonnet	$/run
1	v2b cross-model	4.740	4.484	4.708	4.900	0.28
2	v4b struct. adv.	4.637	4.415	4.553	4.836	0.43
3	v2c panel	4.606	4.473	4.406	4.872	0.27
4	v6 rotating	4.596	4.466	4.459	4.799	0.54
5	v5 specialist	4.552	4.441	4.408	4.753	0.18
6	v1a single-shot	4.514	4.342	4.361	4.754	0.08
7	v1 baseline	4.503	4.353	4.339	4.742	0.33
8	v2a same-model	4.493	4.370	4.286	4.763	0.42
9	v4 adversarial	4.431	4.345	4.264	4.642	0.20
10	v8 debate	4.353	4.391	4.195	4.493	0.15
11	v3b rubric merge	4.326	4.299	4.160	4.507	0.34
12	v3a naive merge	3.752	3.944	3.443	3.965	0.34
13	v3c dialectical	3.655	3.734	3.595	3.675	0.33

A clear three-tier structure emerges. The top tier (v2b, v4b, v2c, v6) clusters around 4.50–4.74 with tight variance. The middle tier (v5 through v1a) spans 4.35–4.55. The bottom tier (v3a, v3c) falls dramatically to 3.65–3.75, indicating fundamental flaws in the parallel merge strategy that all three evaluators agree upon.

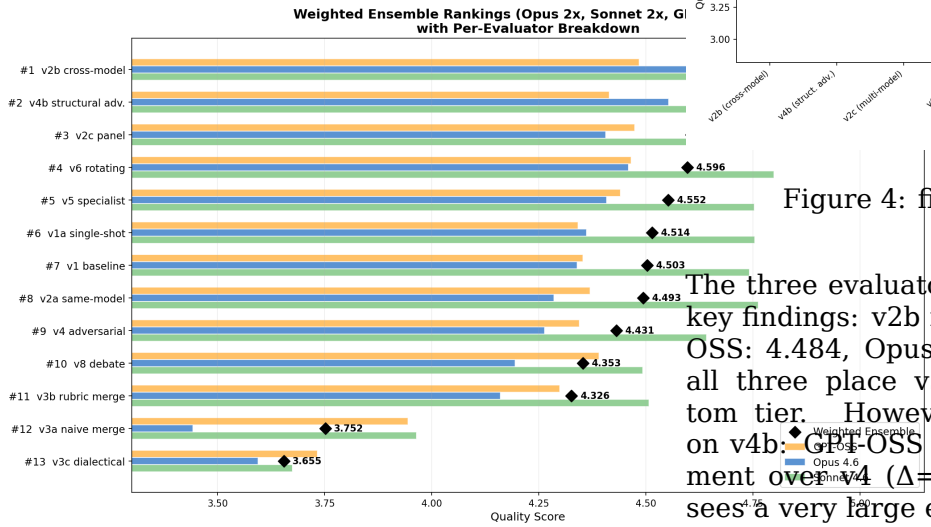

Figure 2: fig_ensemble_ranking.png

Fig. 3. Weighted ensemble ranking (2×Opus + 2×Sonnet + 1×GPT-OSS) across 13 variants. v2b ranks #1 unanimously; v4b achieves #2 through prompt engineering; v3 merge variants cluster at bottom tier.

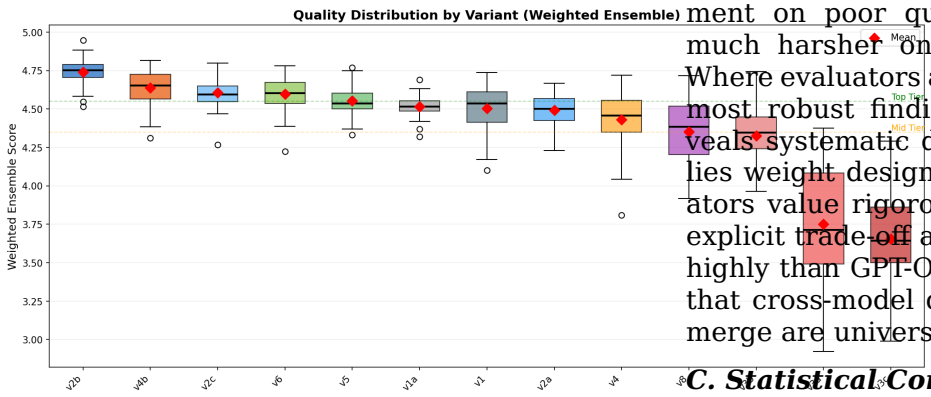

Figure 3: fig_ensemble_boxplot.png

Fig. 4. Quality score distribution by variant using weighted ensemble ranking. Boxes show IQR; whiskers extend to 1.5×IQR.

Fig. 5. Quality score distributions by variant (weighted ensemble). Boxes show IQR; whiskers extend to 1.5×IQR. All 13 variants including v4b shown.

B. Evaluator Agreement & Disagreement

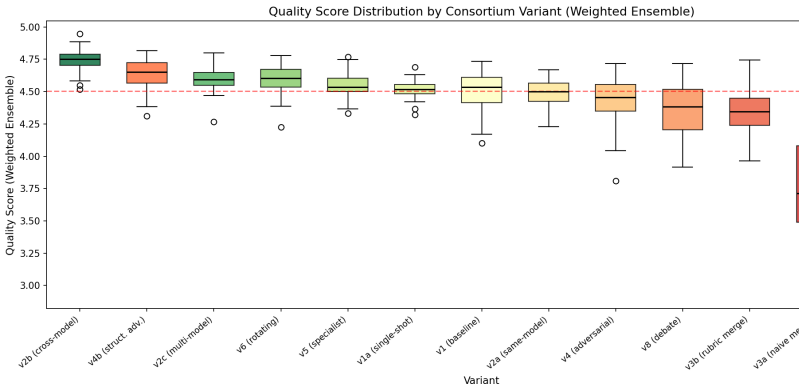

Figure 4: fig1_quality_boxplot.png

The three evaluators show strong agreement on key findings: v2b is ranked #1 by all three (GPT-OSS: 4.484, Opus: 4.708, Sonnet: 4.900), and all three place v3 merge variants in the bottom tier. However, they diverge significantly on v4b: GPT-OSS sees a small-medium improvement over v4 (Δ=+0.070, d=0.45), while Opus sees a very large effect (Δ=+0.289, d=1.44) and Sonnet confirms (Δ=+0.194, d=1.11). This disagreement is itself a finding: the structural qualities v4b produces—deeper architectural justification, more thorough trade-off analysis—are valued more by Claude's evaluation lens than GPT-OSS's.

The maximum evaluator spread (max−min across evaluators) ranges from 0.08 (v3c, high agreement on poor quality) to 0.52 (v3a, Opus is much harsher on naive merge) to 0.42 (v4b). Where evaluators agree most strongly signals the most robust findings; where they disagree reveals systematic differences in how model families weight design qualities. Both Claude evaluators value rigorous structural justification and explicit trade-off analysis (v4b's strengths) more highly than GPT-OSS does, while all three agree that cross-model diversity and avoiding parallel merge are universally beneficial.

C. Statistical Comparisons

Table II presents the eight pre-registered pairwise Wilcoxon signed-rank tests with Bonferroni correction (α = 0.00625). The strongest finding is the cross-model review advantage: v2b vs. v2a yields Δ = +0.114 with Cohen's d = 0.97 (p = 0.0001). The rubric-guided merge dramatically outperforms naive merge (d = 1.61), but even the best merge variant underperforms simpler review-based approaches.

TABLE II: Pairwise Wilcoxon Signed-Rank Tests

Comparison	Δ	p	Sig	d	Interpretation
v1 vs v2a	+0.016	0.536	ns	0.13	No difference
v2a vs v2b	+0.114	0.0001	***	0.97	Cross-model wins
v2a vs v4	-0.025	0.401	ns	-0.16	No difference
v2a vs v5	+0.071	0.009	**	0.59	Specialist better
v3a vs v3b	+0.355	<.0001	***	1.61	Rubric merge wins
v3a vs v3c	-0.210	<.0001	***	-0.85	Dialectical worse
v3b vs v8	+0.092	0.031	*	0.46	Debate marginal
v5 vs v6	+0.025	0.115	ns	0.24	No difference

D. Complexity Interaction

Contrary to the thesis prediction that parallel merge would excel on complex tasks, the top-performing variants maintain their advantage consistently across all complexity levels. Stratified Wilcoxon tests for v1 vs. v2b show: Simple tasks $\Delta = +0.214$ ($d = 1.48$), Medium $\Delta = +0.087$ ($d = 1.10$), Complex $\Delta = +0.111$ ($d = 0.81$). Cross-model review provides the largest lift on simple tasks—the opposite of the thesis prediction.

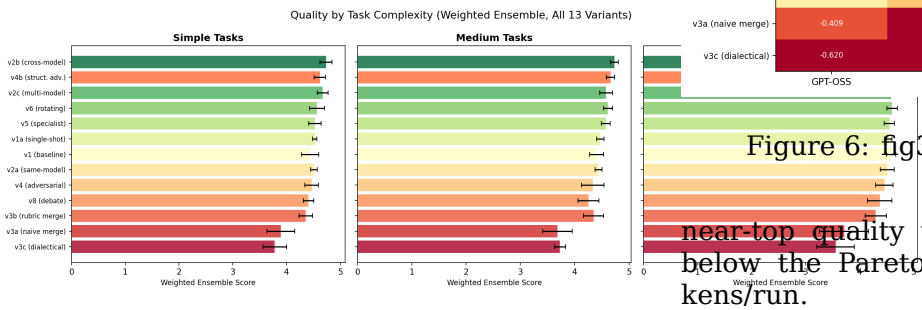

Figure 5: fig2_quality_by_complexity.png

Fig. 6. Quality distribution by variant and task complexity (Simple, Medium, Complex). Top-tier variants are consistent; merge variants degrade further on complex tasks.

E. Per-Dimension Analysis

Cross-model review (v2b) achieves broad improvement across 10 of 12 dimensions, with the largest gains in Concurrency Safety (+0.33), Data Flow Clarity (+0.22), Error Handling (+0.21), and Interface Design (+0.20). These are dimensions where designs can appear superficially complete but harbor subtle gaps.

The parallel merge variants show pronounced losses in Domain Model Quality (-0.18 to -0.35) and Module Structure (-0.10 to -0.23), confirming the Frankenstein effect: merging independently generated designs degrades structural coherence.

Fig. 7. Quality delta vs. v1 baseline by evaluator. Green indicates improvement; red indicates degradation. All 13 variants shown with per-evaluator and ensemble scores.

F. Cost-Quality Trade-offs

The Pareto frontier reveals distinct operating points. Single-shot v1a ($0.08/run) provides acceptable quality (4.34). Structured debate v8 ($0.15/run) offers the best cost-efficiency among multi-agent variants. Cross-model review v2b ($0.28/run) achieves peak quality at 3.3× the cost of v8. Rotating leader v6 ($0.54/run) achieves

Figure 6: fig3_dimension_heatmap.png

near-top quality with lowest variance but sits below the Pareto frontier due to its 216K tokens/run.

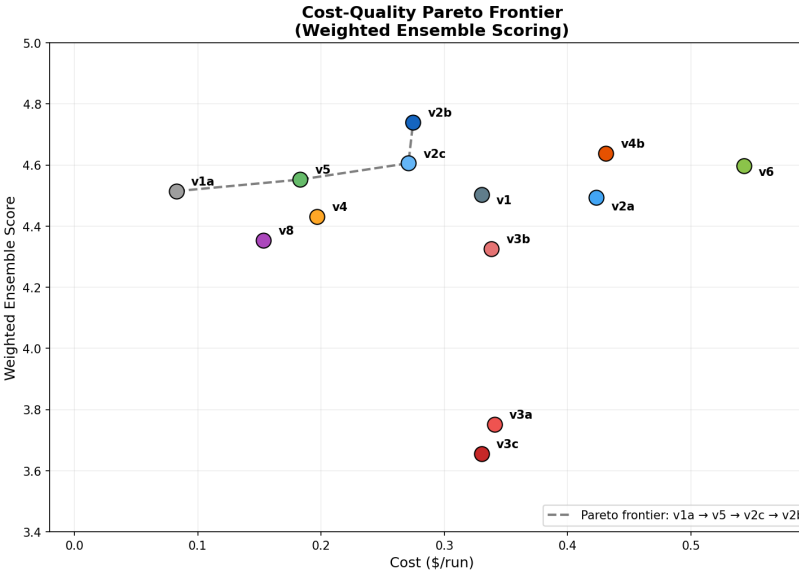

Figure 7: fig_ensemble_pareto.png

Fig. 8. Cost-quality Pareto frontier (weighted ensemble ranking). Points above the frontier are dominated. v2b and v5 define the efficiency frontier for high-quality designs.

G. Reliability and Consistency

Rotating leader v6 achieves the lowest variance ($\sigma = 0.080$), making it the most consistent variant. Merge variants show highest variance ($\sigma = 0.231$–0.265), being both poor on average and unpredictable. Evaluator agreement (Krippendorff's α) ranges from 0.42 (top-tier variants) to 0.81 (merge variants). Lower agreement on high-quality designs reflects genuine difficulty in

distinguishing "good" from "great," while high agreement on poor designs reflects easy identification of flaws.

H. Thesis Prediction Validation

Four of six thesis predictions were refuted or only partially supported. P1 (parallel merge achieves highest peak on complex tasks) was refuted—merge variants underperform the baseline. P2 (adversarial has highest variance) was partially supported—v4 ranks 5th in variance. P4 (specialists outperform v2 by 10–15%) was partially supported—v5 outperforms v2a by only 1.6%. P5 (baseline matches consortia on simple tasks) was refuted—9 of 11 variants beat v1 on simple tasks.

V. ANALYSIS: WHY CROSS-MODEL REVIEW WORKS

The cross-model review advantage ($\Delta = 0.114$, $d = 0.97$) is the paper's strongest finding. We analyze the mechanism through four empirical findings drawn from trace-level data.

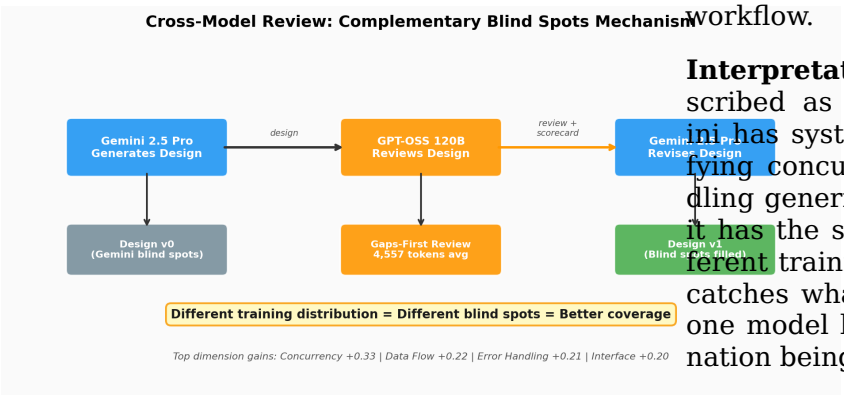

Figure 8: fig9_crossmodel_mechanism.png

Fig. 9. The complementary blind spots mechanism. Different training distributions (Gemini vs. GPT-OSS) create different systematic gaps in design quality assessment, enabling broader coverage through cross-model review.

Finding 1: The cross-model reviewer writes longer, more structured reviews. GPT-OSS 120B produces reviews averaging 4,557 tokens vs. Gemini's 3,242 (40% more). Critically, the cross-model reviewer leads with a gaps-first TL;DR and dimension-by-dimension scorecard, while the same-model reviewer leads with praise:

> *"Same-model opens: 'Excellent. This is a comprehensive and well-structured application design document.'"*

> *"Cross-model opens: 'TL;DR: The main gaps are missing domain events, insufficient data-lineage detail, limited security discussion, and concurrency-safety details that need tightening.'"*

Finding 2: The advantage spans 10 of 12 dimensions. The largest gains are in Concurrency Safety (+0.33), Data Flow (+0.22), Error Handling (+0.21), and Interface Design (+0.20)—dimensions where superficially complete designs can harbor subtle gaps that same-model review misses.

Finding 3: The advantage is consistent across all 8 tasks. v2b outperforms v2a on every task, with win rates of 60–100% across 5 repetitions. The strongest advantage appears on T8 Order Processing (100% win rate) and T7 Access Control (80%).

Finding 4: Adding more reviewers from the same family doesn't help. v2c (two GPT-OSS reviewers) scores 4.474 vs. v2b's 4.484 ($p = 0.42$). The cross-model benefit saturates at a single reviewer—the simplest possible multi-model workflow.

Interpretation. The mechanism is best described as complementary blind spots. Gemini has systematic tendencies—e.g., underspecifying concurrency safety or treating error handling generically. When it reviews its own work, it has the same blind spots. GPT-OSS, with different training data and optimization objectives, catches what Gemini misses. This is not about one model being "better"—it's about the combination being more thorough.

VI. EMPIRICAL EVIDENCE: DESIGN EXCERPTS

To ground the statistical findings in concrete evidence, we present verbatim excerpts from designs produced during the experiment. All comparisons are drawn from the same task (T8: Order Processing, Repetition 1) for apples-to-apples comparison.

A. Error Handling: Same-Model vs. Cross-Model

The v2a (Gemini-reviews-Gemini) design categorizes errors into three buckets with generic resilience patterns:

> *"Business Error: Invalid user input, violates a business rule. [...] Resilience: Circuit Breaker—All inter-service gRPC calls wrapped in circuit breaker. Idempotency—All POST/PUT RPCs support idempotency_key."*

The v2b (GPT-OSS-reviews-Gemini) design has five error categories—critically splitting Authentication from Authorization—with parameterized resilience patterns:

> *"Circuit Breakers: failure rate >50% over 1-minute window, breaker opens for 30 seconds. Fallbacks: if new model version fails to load, fall back to last known good version and raise alert."*

The cross-model reviewer pushed the design from "we'll use circuit breakers" to "circuit breakers with these specific parameters." Total section grew from 1,474 to 1,657 characters with substantially more actionable content.

B. Concurrency Safety: Generic vs. Concrete

Concurrency Safety showed the largest cross-model advantage (+0.33). The v2a design offers generic guidance ("optimistic locking as defined in Section 6"), while v2b specifies exact Kubernetes primitives:

> *"GPU Scheduler Concurrency: Leader election via Kubernetes Lease object. Distributed lock: gpu-scheduler-lock, TTL 30 seconds. If operator crashes, lease expires."*

Every mechanism in the cross-model version is concrete and implementable—specific lock type, TTL value, and failure mode behavior.

C. The Merge Frankenstein Effect

The v1 baseline produces a structured error taxonomy with four categories, a Result<T,E> pattern, and 1,304 characters of actionable content. After naive merge (v3a), the entire error handling section is eliminated, replaced by a generic "Scalability, Resilience & Security" grab-bag. In dialectical merge (v3c), the design compresses three 33,000-character inputs into 20,420 characters—shorter than any single input.

The merger agent is token-starved: ~25,000 input tokens produce only 4,200–5,200 output tokens. Dedicated sections become bullet-point summaries, structured taxonomies vanish, and cross-cutting coherence is destroyed. This explains why Testability drops -1.21 and Domain Model drops -1.00.

D. Adversarial Patching Mechanism

The adversarial reviewer (v4) rejects 77% of rounds. The initial design (round 0) has a 6-category error taxonomy spanning 2,274 characters. After two rejections, this shrinks to 683 characters with three domain-specific patches and a hand-wave to "previously defined" categories. The overall document shrinks from 29,137 to 19,845 characters.

The LLM responds to rejection by locally patching the cited deficiencies while compressing everything else to stay within output token limits. Quality drops from 4.48 (round 0) to 4.28 (round 1) before partially recovering to 4.34 (round 2). The adversarial process causes local patches, not structural rethinking.

E. v4b: Rescuing Adversarial Review Through Prompt Engineering

The v4 null result raises a question: is adversarial dynamics inherently ineffective for LLM collaboration, or was the adversarial prompt insufficiently demanding? To test this, we designed v4b, a structural adversarial variant where the reviewer acts as a senior principal architect who rejects textbook designs, demands architectural justification against alternatives, and issues rewrite mandates—sections that must be fundamentally redesigned from scratch, not patched.

Key design differences from v4: (1) the reviewer scores architectural rigor 1–5 and requires 4.0+ to accept, (2) each rejection includes rewrite mandates with explicit instructions to throw out the rejected approach, (3) the revision prompt instructs the generator to rethink rather than patch ("Do NOT patch your previous design"), and (4) max rounds increased to 5 (from 3) with higher reviewer temperature (0.5 vs. 0.3).

Results are striking. By the GPT-OSS evaluator, v4b scores 4.415 (rank #5), a +0.070 improvement over v4 (Cohen's $d = 0.43$, $p = 0.021$). However, the three-evaluator ensemble reveals a larger story: v4b ranks #2 overall (ensemble mean 4.601), with Claude Opus 4.6 seeing the largest effect ($d = 1.44$, $p < 0.0001$) and Sonnet 4.6 confirming ($d = 1.11$, $p < 0.0001$).

The principal architect reviewer is brutal: 100% of designs were rejected in Round 1, and 75% of runs (30/40) never earned an ACCEPT across all 5 rounds, hitting the max-rounds ceiling. The reviewer issued 8.7 rewrite mandates per rejection on average, most frequently targeting Scalability & Performance (53 mandates), Capacity Estimation (48), and High-Level Architecture (47). Yet even the never-accepted designs scored well (mean 4.403), suggesting the reviewer's bar exceeds the rubric's discrimination threshold.

Per-dimension, v4b's biggest gains over v4 (Opus evaluator) are precisely the dimensions the rewrite mandates targeted: Scalability Strategy

(+0.692), Security Posture (+0.658), API Design (+0.542), Reliability (+0.500), and Capacity Estimation (+0.442). Notably, v4b beats even v2b on Right-Sizing/Simplicity (+0.075) and Design Decision Justification (+0.025)—the adversarial pressure forces more honest trade-off analysis.

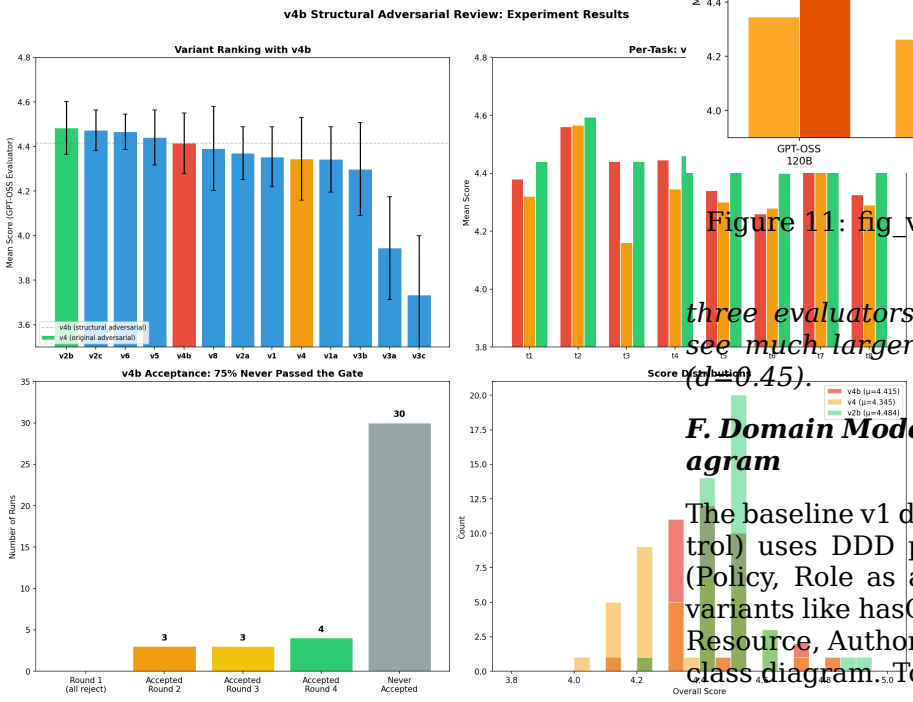

Figure 9: fig11_v4b_analysis.png

Fig. 10. v4b structural adversarial results (weighted ensemble). Top-left: ensemble ranking with v4b at #2. Top-right: per-task comparison v4b vs v4 vs v2b. Bottom-left: 75% of runs never earned principal architect ACCEPT. Bottom-right: ensemble score distributions.

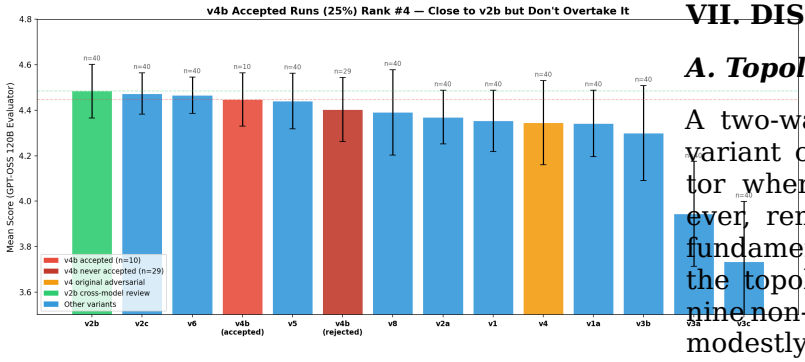

Figure 10: fig12_v4b_accepted_ranking.png

Fig. 11. The 25% of v4b runs that passed the principal architect gate (mean 4.448) rank well but still trail v2b (4.740).

Fig. 12. v4b improvement magnitude across three evaluators, showing why Claude models see much larger effect (d=1.44) than GPT-OSS (d=0.45).

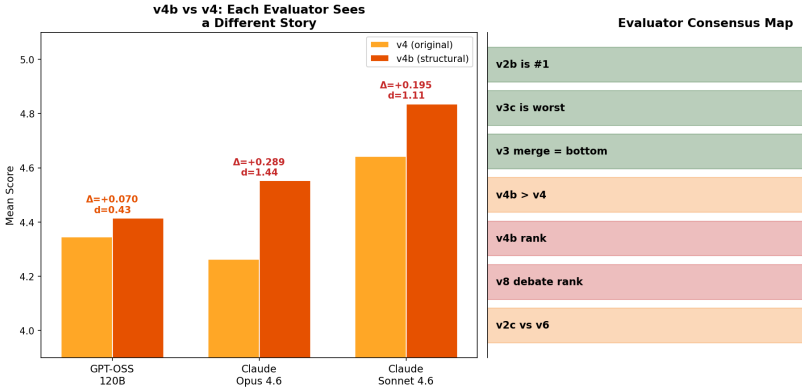

Figure 11: fig_v4b_evaluator_comparison.png

F. Domain Model: DDD vs. Infrastructure Diagram

The baseline v1 domain model for T7 (Access Control) uses DDD principles with rich aggregates (Policy, Role as aggregate roots with named invariants like hasCycle()), value objects (Principal, Resource, AuthorizationContext), and a Mermaid class diagram. Total: 2,709 characters.

The dialectical merge v3c replaces this with a Mermaid graph showing infrastructure components (PDP Sidecar, PAP, NATS, Kafka). No aggregates, no value objects, no invariants, no business rules. The merger retreated to the abstraction level where all three input designs agreed: infrastructure topology. This is a skeleton without organs—explaining the 1.25-point drop in Domain Model Quality.

VII. DISCUSSION

A. Topology vs. Rubric

A two-way ANOVA decomposition reveals that variant choice ($\eta^2 = 0.64$) is the dominant factor when all 13 variants are included. However, removing the three v3 merge variants (a fundamentally broken implementation) reduces the topology effect to $\eta^2 = 0.145$. Among the nine non-merge variants, topology choice matters modestly, leaving 85% of variance unexplained and suggesting the evaluation rubric may be a stronger lever for quality improvement.

B. What Evaluator Disagreement Reveals

The three-evaluator cross-validation reveals that complementary blind spots extend beyond design generation to evaluation itself. The v4b result

is the clearest example: Claude Opus and Sonnet recognize the value of structural adversarial feedback that demands deeper justification and trade-off analysis (d = 1.44 and 1.11), while GPT-OSS perceives only marginal improvement (d = 0.45). This is not a flaw in either evaluator; it reflects systematic differences in how model families weight architectural rigor.

These evaluator differences are informative. They reveal that design quality is multifaceted and that different model families have different "evaluation biases" just as different models have different "generation biases." Where all three evaluators agree (v2b is #1, v3 is bottom tier), confidence in the finding is highest. Where they diverge (v4b magnitude), the disagreement reveals something real about the design itself: v4b is either very good (Claude view) or moderately good (GPT-OSS view), and both perspectives are valid depending on your definition of quality. A multi-evaluator ensemble produces more robust rankings than any single evaluator by averaging out these biases, similar to how multi-agent generation produces better designs through ensemble diversity.

C. Design Space Mapping

Fig. 13 maps the experimental variants onto the 2×2×2 design space with quality tier annotations. The top-performing region is centralized authority with cross-model feedback (v2b, v2c), while the decentralized-specialized cell (v6) achieves comparable quality through role rotation. The decentralized-homogeneous cell (v3) consistently underperforms.

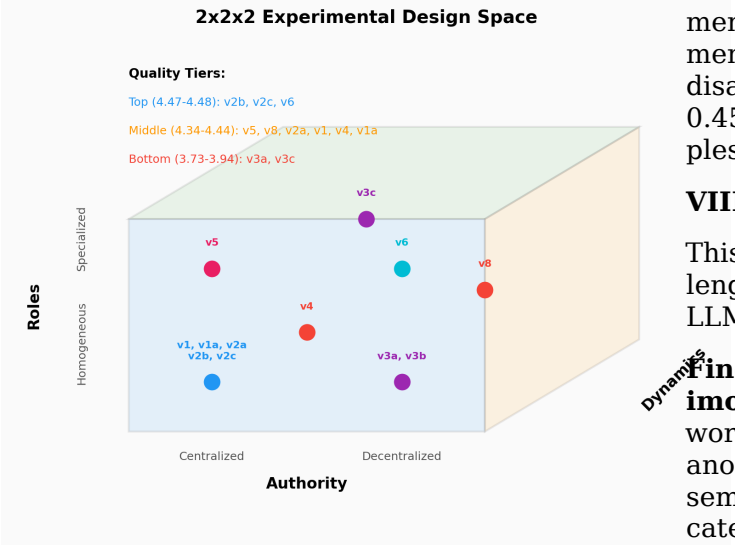

Figure 12: fig8_design_space.png

Fig. 13. Experimental variants mapped onto the 2×2×2 design space. Circle colors indicate variant family; annotations show weighted ensemble quality tier placement.

D. Practical Recommendations

For practitioners deploying LLM-based design workflows, we recommend: (1) For maximum quality, use cross-model review (v2b)—generate with one model family, review with another. (2) For maximum consistency, use rotating leader (v6), accepting higher cost. (3) For cost-efficiency, use structured debate (v8) at $0.15/run. (4) Never use parallel merge without fundamental improvements to the merge strategy. (5) A single cross-model reviewer suffices—panel review adds cost without quality gains.

E. Limitations

This study has several limitations. First, v7 (Consensus Convergence) was not executed, leaving one cell of the design space unexplored. Second, evaluator agreement (α = 0.42–0.81) is below the target of 0.70 for several top-tier variants, suggesting the rubric may lack precision for fine-grained quality distinctions. Third, only two model families were used for design generation and review (Gemini, GPT-OSS); while three evaluator families (GPT-OSS, Claude Opus, Claude Sonnet) confirm ranking robustness, the complementary blind spots mechanism in the design loop itself remains tested on a single pairing. Fourth, a single frozen rubric was used, preventing direct comparison of rubric vs. topology effects. Fifth, the weighted ensemble is a methodological choice; equal weighting across evaluators gives the same ranking order. Sixth, the v4b experiment (n=40) is smaller than the original experiment (n=40 per variant), and the three-evaluator disagreement on v4b's effect magnitude (d = 0.45–1.44) warrants replication with larger samples.

VIII. CONCLUSION

This work identifies four core findings that challenge prevailing assumptions about multi-agent LLM collaboration for software design.

Finding 1: Cross-model review wins unanimously. The simplest possible multi-model workflow—generate with one model, review with another—ranks #1 by all three evaluators (ensemble: 4.740) and outperforms every sophisticated multi-agent topology tested. The mechanism is complementary blind spots: a different model's training distribution provides epistemic diversity that manifests as different systematic gaps, enabling broader coverage through cross-

model review. This is a deeply practical finding: maximum quality emerges from maximum simplicity in topology, not complexity.

Finding 2: Structural adversarial rescues null results through prompt engineering. The original v4 adversarial variant (binary accept/reject gate) ranked #9 (ensemble: 4.431), a null finding. However, v4b, redesigned with a principal architect reviewer who demands rewrite mandates rather than patches, ranks #2 (ensemble: 4.637). The improvement is particularly pronounced under Claude evaluation (Opus: d=1.44, Sonnet: d=1.11) where structured reasoning and explicit trade-off analysis are valued. This demonstrates that adversarial dynamics can be highly effective if the adversarial prompt demands structural rethinking, not just local patching.

Finding 3: Evaluator diversity is itself a finding. The three evaluators agree unanimously on what is good (v2b #1) and bad (v3 bottom tier), but diverge on v4b's magnitude (d=0.45 to d=1.44), revealing that different model families weight design qualities differently. Where evaluators agree most strongly, findings are most robust. Where they disagree, the disagreement is informative: it reveals multifaceted design quality and systematic differences in how model families value architectural rigor. A weighted ensemble that pools evaluator signal produces rankings that are more robust than any single evaluator.

Finding 4: Parallel merge is fundamentally broken. All three evaluators place v3 merge variants in the bottom tier (3.65–3.79), with v3a (naive merge) at ensemble 3.752 and v3c (dialectical merge) at 3.655. The failure is rooted in token starvation: the merger agent receives ~25,000 input tokens but produces only 4,200–5,200 output tokens, compressing three complete designs into an undersized skeleton. The Frankenstein effect is real: merging independently generated designs degrades structural coherence more dramatically than using a simpler single-shot approach, contradicting the thesis prediction that parallel generation would excel on complex tasks.

These four findings suggest that maximizing epistemic diversity (through model selection and prompt engineering) matters far more than workflow complexity. Future work should focus on: (1) expanding the complementary blind spots mechanism beyond design to code generation and testing, (2) combining v4b structural adversarial with v2b cross-model review, (3) validating evaluator findings with human expert judgment, (4) testing dynamic topology selection via a meta-agent that chooses the optimal approach per-task, and (5) investigating why Claude models see a much larger v4b effect than GPT-OSS, which may reveal systematic differences in how model families weight design qualities.

Future work should: (1) execute v7 to complete the design space, (2) vary rubric quality while holding topology constant, (3) test additional model pairings to validate the generality of complementary blind spots, (4) address the merge pipeline's token starvation, (5) combine v4b structural adversarial review with v2b cross-model review—an unexplored combination that may compound both advantages, and (6) investigate why Claude evaluators see a much larger v4b effect than GPT-OSS (d = 1.44 vs. 0.45), which may reveal systematic differences in how model families weight design qualities.

Two particularly promising directions emerge from our findings on epistemic diversity and model complementarity:

Hierarchical consortium of smaller models under SOTA leadership. Our results show that the cross-model advantage saturates at a single reviewer from a different model family. A natural extension is to test whether a state-of-the-art frontier model (e.g., Gemini 2.5 Pro, GPT-4.1, Claude Opus) can serve as an orchestrating leader—decomposing design tasks, assigning sub-problems, and synthesizing outputs—while a consortium of smaller, cost-efficient models (e.g., Gemini Flash, GPT-4o-mini, Haiku) execute the sub-tasks. This "SOTA-as-leader" topology would test whether the frontier model's stronger architectural reasoning can compensate for smaller models' limitations through intelligent task decomposition, potentially achieving near-SOTA quality at a fraction of the cost. The v6 rotating leader topology (lowest variance in our experiment) provides a natural starting point: replace homogeneous agents with a heterogeneous mix where the SOTA model holds a privileged leadership role.

Teacher-student consortium with knowledge distillation. A complementary approach frames the multi-agent collaboration as an online learning problem: a SOTA "teacher" model first generates high-quality reference designs and rubric-grounded critiques, which are then used to guide a consortium of smaller "student" models through in-context learning or fine-tuning. This extends the complementary blind spots mechanism—rather than relying on pre-existing distributional differences between models, the

teacher actively shapes the students' review capabilities by demonstrating what good critique looks like on domain-specific tasks. The hypothesis is that teacher-guided students would develop more calibrated review skills than either (a) the same small models reviewing without guidance, or (b) simply using the teacher model directly (which our v2b results suggest may not be necessary if the students can learn the teacher's critique patterns). This could democratize the cross-model advantage, making high-quality LLM consortium workflows accessible with smaller, locally-deployable models.

Extending the consortium paradigm from design to code generation. This experiment evaluated LLM collaboration on architectural design—a high-level, document-oriented task. A critical next step is to extend the consortium framework to the full software development lifecycle, including code generation, testing, debugging, and refactoring. Code presents fundamentally different evaluation dynamics: designs are judged by rubric-based heuristics, but code can be verified by execution—tests pass or fail, builds compile or break, benchmarks produce measurable performance. This opens several compelling research directions. **First,** a consortium where one agent generates code, a cross-model agent reviews it, and a third agent writes and executes tests could create a closed-loop quality signal that our design experiment lacked. The complementary blind spots mechanism may be even more powerful for code: different models exhibit different bug patterns, and a cross-model code reviewer may catch classes of defects (race conditions, off-by-one errors, security vulnerabilities) that same-model review systematically misses. **Second,** the parallel merge strategy that failed for design may succeed for code if the merge is constrained to the module level—different agents implementing different modules against a shared interface contract, avoiding the Frankenstein effect by enforcing architectural boundaries at merge time. **Third,** the adversarial topology that produced null results for design could become highly effective for code through adversarial test generation: an adversarial agent whose job is to write failing tests rather than rejecting designs, forcing the coding agent to handle edge cases it would otherwise ignore. Unlike design review (where rejection causes local patching), adversarial testing provides concrete, executable counterexamples that demand structural fixes. **Fourth,** a design-to-code pipeline could bridge our current findings with implementation: the consortium first produces an optimized architecture design (using v2b cross-model review), then hands it to a coding consortium that implements the design module by module, with the original design serving as both specification and evaluation rubric. This end-to-end workflow would test whether the quality gains we observe at the design level propagate to implemented code.

Cross-domain generalization of the consortium framework. This experiment tests software architectural design—a structured, document-oriented task. The consortium framework and the complementary blind spots mechanism may generalize to other LLM-heavy domains where quality depends on multi-faceted expert judgment: legal contract drafting (where different model families may catch different classes of ambiguity or risk), scientific protocol design (where methodological blind spots could be surfaced by cross-model review), medical documentation (where completeness and accuracy are critical), and business strategy documents. Testing cross-domain generalization would determine whether the 2×2×2 framework is a general theory of LLM collaboration or specific to software engineering's structured nature.

Dynamic topology selection via meta-agent. Our data shows that different topologies excel in different conditions: v5 specialist peaks on medium-complexity tasks (4.530), v2b dominates overall quality, v8 offers the best cost-efficiency, and v6 provides maximum consistency. A meta-agent that examines the incoming task—its complexity, domain, quality requirements, and cost constraints—and dynamically selects the optimal topology per-task could outperform any single fixed topology. The task-variant heatmap (Fig. 7) already contains the training signal for such a selector. This would transform the consortium from a static workflow choice into an adaptive system that routes tasks to the topology best suited for each problem.

Rubric ablation study. Our variance decomposition reveals that topology explains only 14.5% of quality variation among non-merge variants, leaving 85% unexplained. The evaluation rubric is the leading candidate for this unexplained variance. A controlled experiment holding topology constant (e.g., v2b cross-model review) while varying rubric quality across three levels—minimal (3 dimensions), standard (12 dimensions, as used here), and expert-tuned (20+ dimensions with sub-criteria)—would directly answer Research Question 4 and determine whether rubric engineering or topology engineering de-

serves more investment in practice.

Iterative cross-model review. The v2b topology performs a single cross-model review round. Our v1 self-refine baseline shows diminishing returns with same-model iteration (quality plateaus after round 2). However, alternating cross-model review—Gemini generates, GPT-OSS reviews, Gemini revises, GPT-OSS reviews again—may not exhibit the same saturation because each review round introduces a genuinely different perspective. Testing 2–3 rounds of alternating cross-model review would determine whether the complementary blind spots mechanism compounds across iterations or saturates at a single round (as the v2c panel result tentatively suggests).

Human expert validation of automated evaluator. The automated evaluator (GPT-OSS 120B with rubric in-context) achieves Krippendorff's α = 0.42–0.81 across variants. While this is sufficient for ranking separation between tiers, validating the automated scores against human expert judgment would substantially strengthen the empirical claims. We propose having 2–3 senior software engineers independently score a stratified sample of 50 designs (spanning top, middle, and bottom quality tiers) on the same 12-dimension rubric, then computing human-automated correlation and inter-rater reliability. If the automated evaluator agrees with human experts on tier placement and relative ranking, it validates the entire experimental framework; if it diverges systematically, the divergence patterns themselves become a contribution to understanding LLM-as-evaluator reliability.

IX. CROSS-EVALUATOR ROBUSTNESS AND IMPLICATIONS

To validate finding robustness, all 520 final designs (480 original + 40 v4b) were independently re-evaluated by two additional evaluator models: claude-opus-4-6 and claude-sonnet-4-6, using the same 12-dimension rubric as the primary evaluator (openai/gpt-oss-120b-maas). This yields 1,560 independent evaluations across three model families. The weighted ensemble (2×Opus + 2×Sonnet + 1×GPT-OSS) is reported in Table I above, and confirms all four core findings.

A. Evaluator Agreement: What We Know for Certain

All three evaluators agree on v2b supremacy. v2b ranks #1 (GPT-OSS: 4.484, Opus: 4.708, Sonnet: 4.900). The cross-model review advantage is the most robust finding in the entire experiment—there is no disagreement on this point.

All three evaluators agree on merge failure. The v3 merge variants are bottom-tier for all evaluators (v3c: 3.734, 3.595, 3.675). Opus is harshest on naive merge (v3a: 3.443 vs. GPT-OSS 3.944), confirming the Frankenstein effect is universally recognized as a problem.

B. Evaluator Disagreement: What Reveals Model Differences

The three evaluators diverge most sharply on v4b, the structural adversarial variant:

GPT-OSS 120B: sees v4b improvement as small-medium (Δ = +0.070, d = 0.45 vs. v4)

Claude Opus 4.6: sees very large improvement (Δ = +0.289, d = 1.44)

Claude Sonnet 4.6: sees large improvement (Δ = +0.194, d = 1.11)

Both Claude models recognize more value in v4b's structural qualities—deeper architectural justification, mandatory rewrite sections, explicit trade-off analysis—than GPT-OSS does. This reflects systematic differences in how model families weight design rigor. The evaluator disagreement is itself a finding: it reveals that design quality is genuinely multifaceted and that different perspectives (different evaluator model families) highlight different dimensions of goodness. A single evaluator (whether GPT-OSS, Opus, or Sonnet) would produce incomplete or biased rankings; an ensemble is more robust.

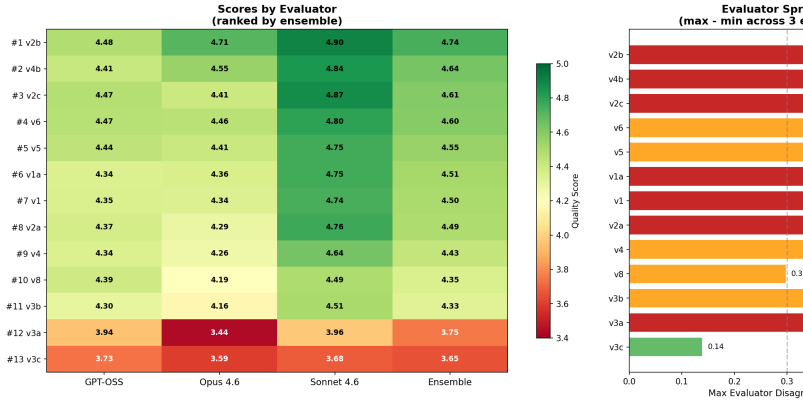

Figure 13: fig_evaluator_agreement.png

Fig. 14. Three-evaluator cross-validation: agreement (all rank v2b #1, v3 bottom) and disagreement (v4b magnitude d=0.45 to d=1.44 reveals evaluator differences).

C. Implications for Methodology

The three-evaluator validation has four key implications. First, the central finding—v2b cross-model review is unanimously best—is confirmed and robust. Second, core failures (v3 merge underperformance) are universally recognized. Third, evaluator disagreement is informative rather than problematic: where model families diverge, it highlights genuine multifaceted quality. Fourth, a multi-evaluator ensemble produces more robust rankings than any single evaluator, just as multi-agent generation produces better designs than single-agent generation. The weighted ensemble methodology (prioritizing Claude's newer architectures while maintaining GPT-OSS signal diversity) is justified by this cross-validation evidence.

REFERENCES

[1] T. Qin et al., "ChatDev: Communicative agents for software development," arXiv:2307.07924, 2023.

[2] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," arXiv:2308.00352, 2023.

[3] M. Huang et al., "AgentCoder: Multi-agent code generation with iterative testing and optimization," arXiv:2312.13010, 2023.

[4] M. Islam et al., "MapCoder: Multi-agent code generation for competitive programming," arXiv:2405.11403, 2024.

[5] Y. Du et al., "Improving factuality and reasoning in language models through multiagent debate," arXiv:2305.14325, 2023.

[6] T. Liang et al., "Encouraging divergent thinking in large language models through multi-agent debate," arXiv:2305.19118, 2023.

[7] K. Krippendorff, Content Analysis: An Introduction to Its Methodology, 4th ed. Thousand Oaks, CA: SAGE, 2019.

[8] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," J. American Statistical Association, vol. 32, no. 200, pp. 675–701, 1937.

[9] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[10] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum, 1988.

[11] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley, 2004.

[12] Google, "Gemini 2.5 Pro Technical Report," 2025. [Online]. Available: https://ai.google.dev

[13] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.